

TD 2 : LES STRUCTURES DYNAMIQUES SEQUENTIELLES (2)

Exercice 1

Ecrire l'algorithme de la fonction **InversListe** (*tête : pliste*) : Pliste qui retourne la tête d'une liste chaînée inverse de la liste chaînée donnée en paramètre.

Prévoir les deux cas : avec et sans copie de valeurs

Exercice 2

Ecrire l'algorithme de la procédure **Eclater** (*tête, var têteInf, var têteSup : Pliste; X : entier*) qui permet de créer à partir d'une liste chaînée donnée et d'un entier X deux listes chaînées de têtes respectives têteInf et têteSup de telle sorte que tous les éléments inférieurs à x seront copiés dans la listeInf et tous les éléments $\geq x$ seront copiés dans la liste listeSup.

Prévoir les deux cas : avec et sans copie de valeurs

Exercice 3

Soient L1 et L2 deux listes chaînées d'entiers triées dans l'ordre croissant, de têtes respectives tête1 et tête2.

Ecrire l'algorithme de la fonction **Fusion** (*tête1, tête2, var têteF : Pliste*) qui permet de créer la liste ListeF à partir des listes L1 et L2, de telle manière que les éléments soient ordonnées selon l'ordre croissant des valeurs.

Prévoir les deux cas : avec et sans copie de valeur

Exercice 4

Soit tête, un pointeur contenant l'adresse du premier élément d'une liste chaînée d'entiers triée dans l'ordre croissant.

Ecrire l'algorithme de la procédure **InsererMilieu** (*var tête : pliste; X : entier*) qui permet d'insérer une valeur X parmi les éléments de la liste en veillant à garder l'ordre croissant des valeurs des Scores.

Exercice 5

Ecrire l'algorithme de la procédure **SupprimDouble** *var* (*tête : Pliste*) qui permet de supprimer les doublons dans une liste chaînée de type LesScores. (garder une seule copie de chaque valeur).

